Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 4

Classification and Regression

Three complete workflows — binary, multiclass, and scalar regression — and the rules for choosing a loss and an output activation that the rest of the book relies on.

Duration 3 hours (2 sessions)

Instructor Rahman Indra Kesuma, S.Kom., M.Cs. · Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 4

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Choose the correct **output activation and loss function** for binary, multiclass, and regression tasks — without guessing.
2. Prepare text with **multi-hot encoding**, and labels as either **one-hot** or **sparse integers**.
3. Read a **validation loss curve** to find the best stopping point, then retrain to exactly that point.
4. Recognise an **information bottleneck** and size intermediate layers from the number of classes.
5. Normalise features using **training-set statistics only**, and evaluate a small-data model with **K-fold cross-validation**.

BOOK
Chapter 4 — full text

NOTEBOOK
01_imdb_binary_classification.ipynb

NOTEBOOK
02_reuters_multiclass.ipynb

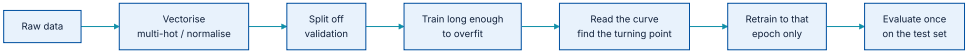
NOTEBOOK
03_housing_regression.ipynb

NOTEBOOK
All chapter 4 notebooks

REPO
Author's official notebook

ROADMAP

The same six steps, three times over



Every example in this chapter follows this path. By the third one it should feel routine.

VOCABULARY

Terms used precisely from here to chapter 20

TERM	MEANING
Sample / input	One data point entering the model.
Prediction / output	What comes out.
Target	The ground truth, from outside the model.
Loss value	A measure of distance between the two.
Mini-batch	Typically 8–128 samples at a time.

TASK TYPE	DEFINING FEATURE
Binary classification	Two mutually exclusive categories.
Multiclass	More than two, one label per sample.
Multilabel	A sample may carry several labels at once.
Scalar regression	One continuous value.
Vector regression	Several continuous values.

VOCABULARY

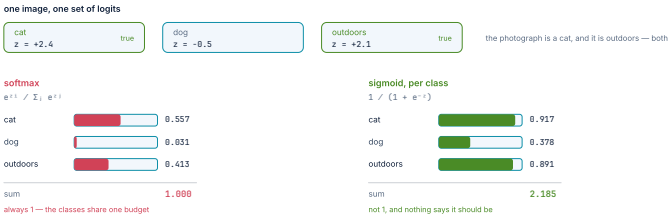
The distinction that decides your output layer

TASK	FINAL LAYER	WHY
Binary — 2 classes	1 unit, **sigmoid**	one number, and 1 − p is the other class
Multiclass — N classes, **one** label	N units, **softmax**	exactly one is true, so they may compete
Multilabel — N classes, **many** labels	N units, **sigmoid**	several may be true, so they must not compete
Scalar regression	1 unit, **no activation**	any real number is a legal answer

Multiclass and multilabel look almost identical on the slide and need different heads. The next slide computes both on the same logits, which is the only way the difference stops being a thing to memorise.

VOCABULARY

. . . and the one line of arithmetic behind it



The same three logits through both heads. Step through them and read the two sums: softmax always makes 1, sigmoid has no reason to.

> The wrong head **cannot express the answer**, and no amount of training data fixes that.

01

Binary classification: IMDB reviews

50,000 movie reviews, positive or negative.

SECTION 4.1

The dataset, and what has already been done to it

listing 4.1 — loading IMDB PYTHON

```python
from keras.datasets import imdb

(train_data, train_labels), (test_data, test_labels) = imdb.load_data(num_words=10000)

print(len(train_data), len(test_data))
print(train_data[0][:12])          # a review, as word indices
print(train_labels[0])             # 1 = positive, 0 = negative
```

OUTPUT

```
25000 25000
[1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468]
1
```

The split is balanced 50:50, which matters for the baseline we set in a moment.

SECTION 4.1

Turning a list of words into a tensor

listing 4.2 — multi-hot encoding PYTHON

```python
import numpy as np

def multi_hot_encode(sequences, num_classes):
    results = np.zeros((len(sequences), num_classes))
    for i, sequence in enumerate(sequences):
        results[i][sequence] = 1.0        # mark every word index that appears
    return results

x_train = multi_hot_encode(train_data, num_classes=10000)
x_test = multi_hot_encode(test_data, num_classes=10000)
y_train = train_labels.astype("float32")
y_test = test_labels.astype("float32")

print(x_train.shape, x_train[0][:12])
```

OUTPUT

```
(25000, 10000) [0. 1. 1. 0. 1. 1. 1. 1. 1. 1. 0. 0.]
```

SECTION 4.1

What that encoding throws away

> Multi-hot records **which words appear**, not **in what order**. *"good, not bad"* and *"bad, not good"* become **the identical vector**

For sentiment on long reviews that turns out to be good enough. Chapters 14 and 15 replace it as soon as word order starts to carry the meaning.

SECTION 4.1

A deliberately small model

listing 4.3 — the model PYTHON

```python
import keras
from keras import layers

model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid"),      # one probability, 0 to 1
])

model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])
```

`relu` zeroes out negatives; `sigmoid` squashes the final score into the interval [0, 1] so it can be read as a probability.

Why crossentropy and not mean squared error

> *Crossentropy is a quantity from the field of information theory that measures the distance between probability distributions.*
>
> Chollet & Watson, section 4.1

The model outputs a **distribution**; the target is a distribution too. Crossentropy is the natural way to compare them, which is why it pairs with sigmoid and softmax rather than MSE.

SECTION 4.1

Holding out a validation set, and training long

listing 4.4–4.5 — validate and fit

PYTHON

```
x_val, partial_x_train = x_train[:10000], x_train[10000:]
y_val, partial_y_train = y_train[:10000], y_train[10000:]

history = model.fit(
    partial_x_train, partial_y_train,
    epochs=20,
    batch_size=512,
    validation_data=(x_val, y_val),
)
```

> Training past the optimum is not waste here — it is **how you find the optimum**. The next slide shows what the curves say.

What the two curves do

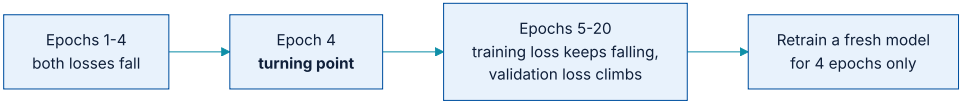
- **Training loss** falls monotonically, epoch after epoch.
- **Validation loss** falls, bottoms out around **epoch 4**, then **turns and climbs**.

> Everything after epoch 4 makes the model **better on data it has seen and worse on data it has not**.

> *After the fourth epoch, you're overoptimizing on the training data, and you end up learning representations that are specific to the training data and don't generalize to data outside of the training set.*
>
> Chollet & Watson, section 4.1

SECTION 4.1

The procedure, stated once and reused everywhere



Overtraining is used as a measuring instrument, then discarded.

SECTION 4.1

Retrain to four epochs, and score it once

the production run

PYTHON

```python
model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.fit(x_train, y_train, epochs=4, batch_size=512)

results = model.evaluate(x_test, y_test)
print(results)
```

OUTPUT

```
[0.2929, 0.8834]
```

88%
test accuracy after retraining to 4 epochs

50%
the random baseline — the classes are balanced

SECTION 4.1

Reading a prediction back

predicting on new reviews

PYTHON

```python
predictions = model.predict(x_test)
print(predictions[:5].ravel())

positive = predictions > 0.5        # the threshold is a CHOICE, not a given
print(positive[:5].ravel())
```

OUTPUT

```
[0.982 0.031 0.671 0.118 0.945]
[ True False  True False  True]
```

Note the third review at **0.67** — confident enough to call positive at a 0.5 threshold, and negative at 0.7. Chapter 6 shows how to set that number **against a cost, not a habit**

02

Multiclass: Reuters newswires

46 topics, mutually exclusive.

SECTION 4.2

Same shape of problem, more classes

listing 4.9 — loading Reuters

PYTHON

```python
from keras.datasets import reuters

(train_data, train_labels), (test_data, test_labels) = reuters.load_data(num_words=10000)

x_train = multi_hot_encode(train_data, num_classes=10000)
x_test = multi_hot_encode(test_data, num_classes=10000)

print(len(train_data), len(test_data), max(train_labels) + 1)
```

OUTPUT

```
8982 2246 46
```

SECTION 4.2

Two ways to write the labels

one-hot labels — PYTHON

```python
from keras.utils import to_categorical

y_train = to_categorical(train_labels)
y_test = to_categorical(test_labels)
# shape (8982, 46)

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"])
```

sparse integer labels — PYTHON

```python
y_train = train_labels      # leave as integers
y_test = test_labels
# shape (8982,)

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

Pick whichever avoids a conversion. Mixing them up gives a shape error that names the loss function, **which is the clue to what went wrong**

SECTION 4.2

The model, and a metric that asks a better question

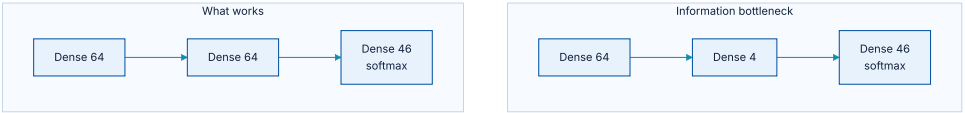
listing 4.11 — model and top-K PYTHON

```python
model = keras.Sequential([
    layers.Dense(64, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(46, activation="softmax"),      # one unit per class
])

top_3 = keras.metrics.TopKCategoricalAccuracy(k=3, name="top_3_accuracy")
model.compile(optimizer="adam",
              loss="categorical_crossentropy",
              metrics=["accuracy", top_3])
```

Top-K accuracy asks whether the true class is among the top k guesses. For a system that **suggests rather than decides**, that is often the honest measure.

SECTION 4.2

An experiment designed to fail



The same network, except one intermediate layer is squeezed to four units.

The book deliberately builds the broken version to show what an **information bottleneck** does. Squeezing 46 classes through 4 dimensions loses information **that no later layer can recover**

SECTION 4.2

What the bottleneck costs

listing 4.13 — the broken model

PYTHON

```python
model = keras.Sequential([
    layers.Dense(64, activation="relu"),
    layers.Dense(4, activation="relu"),          # the bottleneck
    layers.Dense(46, activation="softmax"),
])
```

≈ 80%
validation accuracy without the bottleneck

≈ 71%
with it — nine points gone

> *The model is able to cram most of the necessary information into these 4-dimensional representations, but not all of it.*
>
> Chollet & Watson, section 4.2

SECTION 4.2

The sizing rule that follows

> **An intermediate layer should not be narrower than the number of output classes.** That is why IMDB was fine on 16 units (2 classes) and Reuters needs 64.

It is a floor, not a target. Chapter 5 covers how to find the right size above that floor.

SECTION 4.2

Retrain, evaluate, and check against the right baseline

listing 4.14 — production model

PYTHON

```python
model = keras.Sequential([
    layers.Dense(64, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(46, activation="softmax"),
])
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
model.fit(x_train, y_train, epochs=9, batch_size=512)

print(model.evaluate(x_test, y_test))
```

OUTPUT

```
71/71 ---- 0s 3ms/step - accuracy: 0.7969 - loss: 0.9127
[0.9127, 0.7969]
```

80% sounds unremarkable until you know the baseline: guessing according to the class frequencies gets about **19%**, because the 46 classes are **far from evenly distributed**

SECTION 4.2

Where the remaining 20% goes

inspecting the errors PYTHON

```python
predictions = model.predict(x_test)
predicted = predictions.argmax(axis=1)

print((predicted == test_labels).mean())          # overall accuracy
print(predictions[0].max())                       # confidence on the first one

import collections
print(collections.Counter(test_labels).most_common(3))   # class imbalance
```

OUTPUT

```
0.7969
0.94
[(3, 813), (4, 474), (19, 133)]
```

Class 3 alone accounts for **813 of 2,246** test samples. That imbalance is why the honest baseline is 19% rather than 1/46 — and why **accuracy alone is a thin way to describe this model**

03

Scalar regression: house prices

Very little data. The rules change.

SECTION 4.3

480 training samples, eight features

listing 4.16 — loading the data PYTHON

```python
from keras.datasets import california_housing

(train_data, train_targets), (test_data, test_targets) = (
    california_housing.load_data(version="small"))

print(train_data.shape, test_data.shape)
print(train_targets[:4])
```

OUTPUT

```
(480, 8) (120, 8)
[452600. 358500. 352100. 341300.]
```

Features: longitude, latitude, median house age, population, households, median income, total rooms, total bedrooms.

SECTION 4.3

Normalisation — and the leak hiding in it

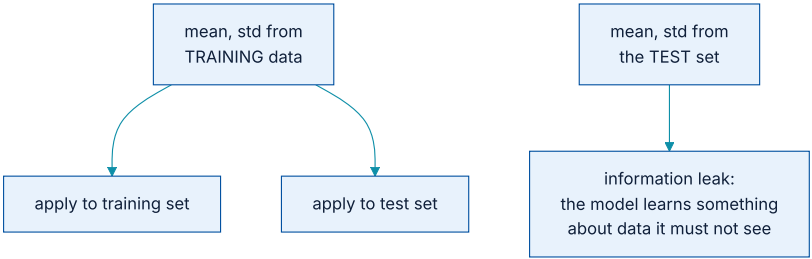
listing 4.17 — normalising PYTHON

```
mean = train_data.mean(axis=0)
std = train_data.std(axis=0)

x_train = (train_data - mean) / std
x_test = (test_data - mean) / std      # TRAINING statistics, not the test set's

y_train = train_targets / 100000       # scale the target to a sane range
y_test = test_targets / 100000
```

Recomputing `mean` and `std` from the test data would be an **information leak**: the model would know something about data it is supposed never to have seen. Chapter 5 gives this its formal name.

SECTION 4.3

Which statistics may touch which data



The rule in one picture: statistics flow outward from the training set only.

SECTION 4.3

A regression head has no activation

listing 4.18 — the model PYTHON

```python
def get_model():
    model = keras.Sequential([
        layers.Dense(64, activation="relu"),
        layers.Dense(64, activation="relu"),
        layers.Dense(1),                    # NO activation — free to predict any value
    ])
    model.compile(optimizer="adam",
                  loss="mean_squared_error",
                  metrics=["mean_absolute_error"])
    return model
```

No output activation

A sigmoid would trap predictions in [0, 1]. Regression must be free.

A small model

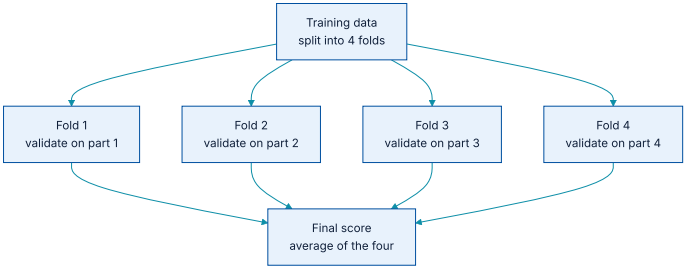
480 samples. A large model would simply memorise them.

MSE to train, MAE to read

MAE is human-readable: 0.5 means being off by about $50,000.

SECTION 4.3

With this little data, one validation split is not enough



Each quarter takes a turn as the validation set; the four scores are averaged.

With 480 samples, a single split of, say, 120 is small enough that the score depends heavily on **which 120 you happen to draw**

SECTION 4.3

K-fold, written out

listing 4.19 — the K-fold loop PYTHON

```python
k, num_epochs, all_scores = 4, 50, []
num_val_samples = len(x_train) // k

for i in range(k):
    fold_x_val = x_train[i * num_val_samples : (i + 1) * num_val_samples]
    fold_y_val = y_train[i * num_val_samples : (i + 1) * num_val_samples]
    fold_x_train = np.concatenate(
        [x_train[: i * num_val_samples], x_train[(i + 1) * num_val_samples :]], axis=0)
    fold_y_train = np.concatenate(
        [y_train[: i * num_val_samples], y_train[(i + 1) * num_val_samples :]], axis=0)

    model = get_model()                # a FRESH model every fold
    model.fit(fold_x_train, fold_y_train, epochs=num_epochs, batch_size=16, verbose=0)
    val_loss, val_mae = model.evaluate(fold_x_val, fold_y_val, verbose=0)
    all_scores.append(val_mae)
```

OUTPUT

```
[0.265, 0.292, 0.232, 0.349]
mean MAE: 0.296   ->  off by roughly $29,600
```

SECTION 4.3

The spread is the point

> The four folds scored **0.232 to 0.349** — a spread of nearly 50% of the smallest value. Any single split would have reported one of those numbers and **told you nothing about the other three**

That variance is exactly why K-fold exists, and why it is worth four times the compute on a dataset this size.

SECTION 4.3

How long to train? Average the curves

listing 4.20 — averaging MAE curves PYTHON

```python
all_mae_histories = []
for i in range(k):
    # ... same fold setup as before ...
    history = model.fit(fold_x_train, fold_y_train,
                        validation_data=(fold_x_val, fold_y_val),
                        epochs=200, batch_size=16, verbose=0)
    all_mae_histories.append(history.history["val_mean_absolute_error"])

average_mae_history = [
    np.mean([h[i] for h in all_mae_histories]) for i in range(200)
]
```

The averaged curve flattens around **epoch 120–140** and worsens after. **That is the stopping point**, and it is far more trustworthy than any single fold's curve.

SECTION 4.3

The final model, and what its error means

the production model

PYTHON

```python
model = get_model()
model.fit(x_train, y_train, epochs=130, batch_size=16, verbose=0)

test_mse, test_mae = model.evaluate(x_test, y_test)
predictions = model.predict(x_test)
print(f"test MAE {test_mae:.2f}   first prediction {predictions[0][0]:.2f}")
```

OUTPUT

```
test MAE 0.31   first prediction 2.83
```

≈ $31,000
average error, at the 100k scaling

≈ $283,000
what a prediction of 2.83 means

ACROSS ALL THREE EXAMPLES

Why every model here was kept small

EXAMPLE	TRAINING SAMPLES	HIDDEN LAYERS	REASONING
IMDB	15,000	2 × 16 units	Two classes, so 16 units is comfortably above the floor.
Reuters	8,982	2 × 64 units	46 classes: anything much narrower becomes a bottleneck.
Housing	480	2 × 64 units	Tiny dataset — a larger model would memorise it outright.

The rule pulls in two directions at once: **wide enough not to bottleneck the classes, small enough not to memorise the samples**. Chapter 5 turns that tension into a method.

ACROSS ALL THREE EXAMPLES

Four ways these workflows go wrong

Training to a fixed epoch count

Picking 20 epochs because a tutorial did. The turning point is a property of **your** data and has to be measured.

Reusing a trained model across folds

`get_model()` must be called **inside** the K-fold loop. Reusing one carries knowledge between folds and voids the score.

Normalising with test statistics

An information leak, and it inflates your score in a way you will not discover until production.

Reporting accuracy with no baseline

80% is excellent against 19% and poor against 90%. The number alone says nothing.

SUMMARY

The table you will keep coming back to

TASK	LAST-LAYER ACTIVATION	LOSS FUNCTION	TYPICAL METRICS
Binary classification	`sigmoid` (1 unit)	`binary_crossentropy`	accuracy, ROC AUC
Multiclass, one-hot labels	`softmax` (N units)	`categorical_crossentropy`	accuracy, top-K
Multiclass, integer labels	`softmax` (N units)	`sparse_categorical_crossentropy`	accuracy
Multilabel	`sigmoid` (N units)	`binary_crossentropy`	accuracy, ROC AUC
Scalar regression	none (1 unit)	`mean_squared_error`	MAE

SUMMARY

What has to survive this chapter

1. **Vectorise** discrete data; **normalise features using training-set statistics only**.
2. **Intermediate layers must not be narrower than the class count** — otherwise you build an information bottleneck.
3. **Little data → a small model**, one or two hidden layers.
4. **Train until it overfits to find the turning point**, then retrain to exactly that epoch.
5. **Little data → K-fold**, not a single validation split — and read the spread, not just the mean.
6. Always compare against a **common-sense baseline** before deciding a score is good.

NOTEBOOK

01_imdb_binary_classification.ipynb

NEXT

Chapter 5 — Fundamentals of machine learning